

Vinci Unitree Day 2 Lecture Notes

Course: Vinci Unitree Training • Day 2 — Go2 Autonomy & Sandbox Capstone

Lecture duration: 3 hours

Platform: Unitree Go2

Primary stack: `unitree_sdk2_python` , CycloneDDS, Python SDK clients, optional ROS 2/Gazebo context

Prepared by: Manus AI

Version: 2026-06-03

Classification: Property of Vinci AI — Do Not Distribute

Copyright: © 2026 Vinci AI. All rights reserved.

Instructor framing. Day 2 is not a promise of full autonomy. It is a disciplined transition from Day 1 robot connectivity and single-action control into a bounded inspection workflow: define a scenario, validate data products, move safely under local obstacle avoidance, capture checkpoint evidence, tune the plan, and report what happened. The central mental model is **sense → log → decide → act → report**.

1. Lecture Purpose and Learning Outcomes

Day 2 extends Day 1’s low-risk primitives into an inspection patrol pipeline. Students should leave the lecture able to explain how a Go2 inspection run is represented as files, how `ObstaclesAvoidClient` is used for local avoidance and incremental legs, how checkpoint images and DDS state logs become evidence, and why the course deliberately distinguishes open-loop patrol from true SLAM/navigation.

By the end of the three-hour lecture, students should be able to describe the full Day 2 workflow, identify the role of every major script in `course/day-02/` , read a `patrol_plan.json` , interpret a `metadata.json` and `sportmodestate.jsonl` , explain the difference between **reactive obstacle avoidance**, **open-loop increment patrol**, **simulation**, and **SLAM/navigation**, and apply the safety rules that govern field operation. These outcomes align with the repository’s Day 2 objective: turning Day 1 skills into inspection autonomy through SLAM/planning concepts, obstacle avoidance, sensor capture, optional simulation, field patrol, and team presentation.¹

Outcome area	Student should be able to explain	Evidence of understanding
System architecture	How Python scripts use DDS/RPC clients to control Go2	Student can draw PC → DDS/RPC → Go2 → run folder.

	and collect evidence.	
Inspection data	Why <code>metadata.json</code> , <code>patrol_plan.json</code> , <code>sportmodestate.jsonl</code> , and checkpoint images are separated.	Student can validate or diagnose a run folder.
Motion semantics	Difference between velocity streaming and increment goals.	Student can read a leg entry and predict robot behaviour.
Safety and limits	Why speeds, increments, cones, spotters, and cleanup are required.	Student can name abort conditions before touching hardware.
Autonomy boundaries	Why Day 2 patrol is not GPS, not map-based navigation, and not full SLAM.	Student can distinguish local avoidance from mapping/localization/planning.

2. Three-Hour Teaching Plan

The repository maps Day 2 across a full training day, with a morning SLAM/fusion lecture, a planning/avoid practical, an afternoon Gazebo sandbox, and a field patrol/presentation block.² For a three-hour lecture, the best structure is to compress the full-day arc into a concept-rich lecture with short demonstrations and guided code/data walkthroughs. The lecture should not attempt to complete every hardware lab live; instead, it should prepare students to perform the labs safely and intelligently.

Time	Segment	Main teaching objective	Instructor emphasis
00:00–00:15	Day 1 recap and Day 2 framing	Connect Day 1 topics to patrol autonomy.	“We are assembling primitives into a controlled inspection workflow.”
00:15–00:35	Inspection architecture	Introduce sense → log → decide → act → report.	Show why autonomy is also data discipline, not only movement.
00:35–01:00	Readiness, scenario, and safety	Explain <code>my_team_scenario.json</code> , limits, and abort rules.	Make speed caps and cleanup non-negotiable.
01:00–01:30	Run-folder schema and validation	Explain <code>metadata.json</code> , <code>patrol_plan.json</code> ,	Treat the run folder as the inspection

		JSONL logs, checkpoint images.	deliverable.
01:30–01:40	Break	Reset attention before motion/API topics.	Confirm students understand open-loop vs map-based autonomy.
01:40–02:10	Obstacle avoidance and local motion APIs	Teach <code>ObstaclesAvoidClient</code> lifecycle, <code>Move</code> , and <code>MoveToIncrementPosition</code> .	Emphasize <code>UseRemoteCommandFromApi(True)</code> and repeated/controlled command sending.
02:10–02:35	Multi-leg patrol and integrated capture	Walk through patrol plan execution and <code>lab03_patrol_runner.py</code> .	Explain start capture, leg execution, dwell, checkpoint capture, metadata.
02:35–02:50	Gazebo and ROS 2 context	Contrast simulation <code>/cmd_vel</code> with hardware SDK clients.	Use sim as design validation, not a substitute for field safety.
02:50–03:00	Field trial, tuning, capstone, and knowledge check	Close with tuning/reporting and presentation expectations.	Every run must end as evidence, not just a console PASS.

3. Conceptual Foundation: What “Inspection Autonomy” Means on Day 2

In this course, inspection autonomy means a robot follows a bounded, human-prepared patrol routine in a cleared arena, logs state while moving, captures visual evidence at named checkpoints, and produces a run folder that a human team can inspect and present. It does **not** mean that the robot builds a global map, localizes within that map, plans a global path, recognizes cones visually, or makes unsupervised decisions in an unknown environment.

This distinction matters because robotics vocabulary is often overloaded. In general robotics, **SLAM** refers to building or updating a map while estimating the robot’s pose in that map, and navigation stacks such as Nav2 separate concerns into behavior trees, planners, controllers, recovery behaviours, lifecycle nodes, and long-running navigation actions.³ Day 2 introduces these ideas conceptually, but the main Python patrol labs use

local obstacle avoidance and scripted increments. The students are learning a reliable engineering workflow before they graduate to map-based navigation.

Definition for class. An inspection patrol is a **scripted, evidence-producing mission** in which checkpoint IDs, motion legs, safety limits, captures, logs, and report fields are all explicit before the robot moves.

Term	Meaning in robotics	Meaning in Day 2 lecture
Reactive obstacle avoidance	The robot responds locally to nearby obstacles while accepting velocity or increment commands.	Used directly through <code>ObstaclesAvoidClient</code> .
Open-loop patrol	A sequence is executed without closed-loop correction to a semantic target.	The default Day 2 patrol model: legs come from <code>patrol_plan.json</code> .
SLAM	Simultaneous mapping and localization.	Conceptual background and optional advanced context, not the core Python patrol.
Navigation stack	Integrated map, localization, planner, controller, behaviours, and goals.	Mentioned to explain what Day 2 is not yet doing.
Inspection evidence	Data that proves what was run, where the robot stopped, and what it saw.	The run folder: metadata, plan, JSONL state, images, field report.

4. Day 1 Recap: The Primitives Day 2 Reuses

Day 2 assumes the students have already established the essential Go2 development loop: connect the PC to the Go2 network, initialize DDS communication, read state, prepare a safe posture, call high-level sport motion services, and capture a basic inspection image. These are not isolated Day 1 exercises; they are the building blocks of Day 2 patrol.

The repository explicitly maps Day 1 skills into Day 2 uses. Subscribing to `rt/sportmodestate` becomes patrol logging. `SportClient` , `CheckMode` , stand-up, and balance stand become the pre-motion readiness sequence. `VideoClient` becomes checkpoint evidence capture. The obstacle avoidance introduction becomes the foundation for multi-leg patrol.⁴

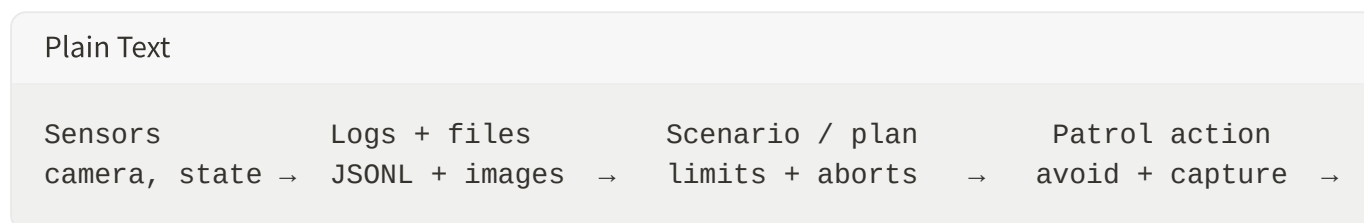
Day 1 capability	Day 2 reuse	Why it matters
------------------	-------------	----------------

DDS topic subscription	<code>sportmodestate.jsonl</code> during patrol.	The team can reconstruct motion state and error codes.
Sport state and posture checks	Stand preparation before patrol.	The robot should not enter avoid/patrol from an unsafe posture.
Single checkpoint image capture	<code>checkpoints/<id>/frame.jpg</code> .	Visual evidence becomes tied to a patrol plan.
Obstacle avoidance preview	Increment and velocity patrol legs.	Students move from one short motion to a planned sequence.
Clean shutdown habits	Patrol cleanup on success, failure, or <code>Ctrl+C</code> .	The robot must always stop and release API control.

Instructor note: if a team did not pass Day 1 motion on hardware, they should not treat Day 2 as a way to “try again faster.” They may continue schema, validation, simulation, and dry-run work, but real movement should wait until Day 1 readiness and field safety are restored.

5. Inspection Architecture: Sense → Log → Decide → Act → Report

The most useful single diagram for the lecture is the inspection pipeline. Sensors provide state and images. The software logs DDS state and camera captures. The scenario and plan encode what the team intends to do. The patrol code acts through the avoidance client. The final run folder becomes the reportable evidence.



This pipeline makes inspection autonomy auditable. If the robot stops short of a cone, the team should be able to inspect the plan, compare baseline and tuned runs, view checkpoint images, read JSONL state, and explain what changed. The goal is not merely to make the dog walk; the goal is to create a reproducible and reviewable inspection run.

Pipeline stage	Day 2 implementation	Typical question students should answer
----------------	----------------------	---

Sense	VideoClient , rt/sportmodestate , optional platform probe.	What did the robot see, and what state was it in?
Log	sportmodestate.jsonl , checkpoint frame.jpg , optional state_slice.jsonl .	Is there enough data to reconstruct the run?
Decide	Scenario card, speed limits, abort rules, plan legs.	Why is this motion allowed, and when should it stop?
Act	ObstaclesAvoidClient , increment/velocity legs.	Which API command moves the robot, and in which frame?
Report	metadata.json , field_test.md , validator PASS/FAIL.	What evidence proves the run was valid?

6. Readiness and Scenario Definition

The first Day 2 lab is intentionally conservative. It asks teams to confirm that Day 1 artifacts exist, that the patrol-related SDK imports are available, that network and DDS readiness are acceptable, and that a patrol scenario has been written and validated before motion begins. This is the correct order: define the mission and limits first, then move the robot.

A scenario file is not paperwork. It is the safety and mission contract for the day. The scenario defines the team, operator, arena, hazards, checkpoints, motion limits, abort rules, and required deliverables. In the repository, the class caps are especially important: forward velocity should remain at or below **0.25 m/s**, and increment distance should remain at or below **0.5 m** unless the instructor approves otherwise.⁴

Scenario field	Lecture explanation	Common mistake to prevent
team_name / operator	Identifies who owns the run and report.	Leaving metadata anonymous.
arena	Describes physical boundaries, floor, and hazards.	Treating any open space as acceptable.
checkpoints	Names the stops where evidence will be captured.	Using inconsistent IDs between scenario and plan.
motion_limits	Defines allowed vx , dx , dy , and yaw rates/angles.	Copying SDK maximums instead of class limits.
abort_rules	States when the spotter or script must halt.	Writing vague rules that cannot be acted on.

deliverables

Defines what must be submitted.

Ending with only a terminal transcript.

A useful instructor prompt is: “If I hand your scenario file to another team, could they understand the arena, checkpoints, speed limits, and abort conditions without asking you?” If the answer is no, the scenario is not yet ready for motion.

7. Safety Rules for Day 2 Patrol

Safety must be taught as part of the software architecture. Day 2 adds multi-stop motion, so the risk is no longer only whether a single command works; the risk is whether a sequence continues after conditions change. The repository’s Day 2 safety rules require a marked arena boundary, one patrol at a time per Go2 subnet, default speed and increment caps, obstacle avoidance enabled for patrol legs, clean shutdown, local increment framing, and no acrobatics.[4](#)

Rule	Required behaviour	Rationale
Marked arena	Corners are marked with cones and the stop caller is agreed before motion.	Everyone knows the physical operating envelope.
One patrol at a time	Only one team commands a Go2 on the subnet.	Prevents command confusion and network contention.
Speed cap	Keep <code>vx <= 0.25 m/s</code> unless approved.	Preserves reaction time and reduces impact risk.
Increment cap	Keep <code>dx <= 0.5 m</code> unless approved.	Prevents large open-loop jumps.
Avoid mode default	Use <code>SwitchSet(True)</code> and <code>UseRemoteCommandFromApi(True)</code> .	Ensures commands flow through the avoid service.
Clean shutdown	Send zero motion, release API command source, disable avoid.	Avoids lingering command authority.
No SLAM claims	Describe increments as local, not GPS or global navigation.	Prevents false mental models.

No acrobatics	Exclude flips/handstands from patrol paths.	Keeps the day focused on inspection safety.
---------------	---	---

Instructor safety sentence. “The robot may be capable of higher speeds and larger increments than we use today, but class limits are chosen for supervision, evidence quality, and repeatability, not for demonstrating the robot’ s maximum capability.”

8. Run-Folder Schema: The Inspection Deliverable

The run folder is the central data product of Day 2. It transforms a robot demonstration into an auditable inspection artifact. A valid run folder contains metadata, a patrol plan, a state log, and checkpoint-specific files. The validator enforces enough structure that students learn to treat field robotics as data engineering as well as control.

Plain Text

```
run_YYYYMMDD_HHMM/
  metadata.json
  patrol_plan.json
  sportmodestate.jsonl
  checkpoints/
    cp_A/
      frame.jpg
      state_slice.jsonl
    cp_B/
      frame.jpg
    cp_C/
      frame.jpg
```

Compared with a Day 1 single-checkpoint bundle, the Day 2 run folder adds multiple checkpoints, multiple legs, a formal plan, and a fuller state log. `metadata.json` tells the reader who ran the mission, when it was created, which interface and check mode were used, which checkpoints exist, and which artifacts should be present. `patrol_plan.json` tells the robot and the human reviewer what the intended checkpoint sequence and motion legs were. `sportmodestate.jsonl` records state samples line by line, which is convenient for streaming logs and robust partial writes.⁵

File	Required role	Instructor explanation
<code>metadata.json</code>	Run identity and artifact index.	“Who ran what, when, on which interface, with which robot state?”

patrol_plan.json	Mission and motion definition.	“What was supposed to happen?”
sportmodestate.jsonl	Time-series state evidence.	“What state did the robot report during the run?”
checkpoints/<id>/frame.jpg	Visual evidence at a stop.	“What did the robot see at this checkpoint?”
state_slice.jsonl	Optional local context around capture.	“What was the state near this image?”
field_test.md	Field trial interpretation.	“What changed, what passed, what still needs tuning?”

The validator lab should be taught as a diagnostic exercise, not as a mechanical command. Students should intentionally validate a passing fixture and an incomplete fixture. The point is to train them to read failure messages as structured evidence of missing artifacts, inconsistent checkpoint IDs, invalid JSONL lines, or absent images.

9. patrol_plan.json : How a Patrol Becomes Executable

The patrol plan is the bridge between a human scenario and executable robot motion. It contains checkpoint definitions and leg definitions. Checkpoints provide IDs, labels, and dwell times. Legs provide movement from one checkpoint toward another, using either an increment command or a timed velocity command.

The most important lecture distinction is between **velocity motion** and **increment motion**. A velocity leg says “command this body velocity for this duration.” An increment leg says “ask the avoidance service to move this local body-frame increment.” Neither statement is the same as “navigate to the blue cone” unless another perception and localization layer closes that loop.

Leg type	Command relationship	Meaning	Day 2 use
increment	MoveToIncrementPosition (dx, dy, dyaw)	Move a local body-frame increment under avoid mode.	Default multi-leg patrol.
velocity	Repeated Move(vx, vy, vyaw) for a duration.	Stream body velocity for a fixed time.	Optional advanced tuning or comparison.
dwell or dwell_s	Sleep at checkpoint.	Stop long enough to settle and capture.	Used before images and between legs.

A default L-shaped cone course is used to keep the geometry visible. The dog starts at `cp_A`, walks a short forward increment toward `cp_B`, turns at `cp_B`, then walks forward toward `cp_C`. The example turn of approximately `0.6 rad` is about 34 degrees, and the straight increments are deliberately short. Students should understand that tuning `dx` and `dyaw` changes the open-loop behaviour; it does not cause the robot to recognize or steer toward the cone.⁶

10. Python SDK and Unitree Context

Unitree's SDK documentation describes `unitree_sdk2` as an SDK for new-generation robots that exposes interfaces for low-level motor control, high-level motion control, LiDAR point cloud data, audio/video transmission, SLAM, odometry, and other functions.⁷ The repository's Day 2 materials use the Python SDK path for Go2 inspection patrol, not a full ROS navigation stack.

The network prerequisites should be stated clearly. Unitree's quick-start documentation recommends developing on Ubuntu 20.04 or 22.04, placing the user PC's robot-facing network adapter on the `192.168.123` subnet, not assigning the PC the robot onboard address `192.168.123.161`, and testing connectivity by pinging `192.168.123.161`.⁸ In the course materials, students commonly pass an interface name such as `en6` into scripts. The exact interface varies by laptop and USB Ethernet adapter.

Concept	Course interpretation
<code>ChannelFactoryInitialize(0, "en6")</code>	Initialize SDK communication on the robot-facing interface.
<code>SportClient</code>	Prepare posture and stop sport movement when needed.
<code>MotionSwitcherClient.CheckMode()</code>	Record current mode in metadata and readiness checks.
<code>ObstaclesAvoidClient</code>	Enable local obstacle avoidance and execute avoid-mode motion.
<code>VideoClient</code>	Pull camera frames for checkpoint evidence.
<code>ChannelSubscriber("rt/sportmodestate", SportModeState_)</code>	Subscribe to state and write JSONL logs.

11. ObstaclesAvoidClient : Lifecycle and Semantics

The Day 2 patrol uses `ObstaclesAvoidClient` because it provides the Go2 obstacle avoidance service interface. Unitree's documentation states that `SwitchSet` enables or disables obstacle avoidance, `SwitchGet` reads whether avoidance is enabled,

`UseRemoteCommandFromApi(True)` takes over speed command control from the remote controller, `Move(x, y, yaw)` controls velocity in obstacle avoidance mode, and `MoveToIncrementPosition(x, y, yaw)` controls incremental movement in the body frame.⁹

The lifecycle is as important as the motion command itself. A safe script initializes communication, prepares posture, enables obstacle avoidance, transfers command authority to the API, sends movement commands, sends zero motion at the end, returns command authority, disables obstacle avoidance, and stops sport movement if needed. This sequence is encoded in the repository helper functions `enable_avoid()` and `release_avoid()`.¹⁰

Step	API action	Teaching explanation
Initialize	Create client, set timeout, <code>Init()</code> .	The script must bind to the robot service before commanding.
Enable avoid	<code>SwitchSet(True)</code> and verify with <code>SwitchGet()</code> .	Avoid mode must actually be on, not assumed.
API command source	<code>UseRemoteCommandFromApi(True)</code> .	Unitree documentation requires this for API avoid control.
Move	<code>Move(...)</code> or <code>MoveToIncrementPosition(...)</code> .	Commands are local body-frame velocity or increment requests.
Stop	Send repeated <code>Move(0,0,0)</code> .	Stop commands should be explicit and redundant.
Release	<code>UseRemoteCommandFromApi(False)</code> , <code>SwitchSet(False)</code> .	The script must not retain control after completion.
Sport cleanup	<code>SportClient.StopMove()</code> when available.	Adds another layer of stop semantics.

The helper implementation sends repeated zero `Move` commands during cleanup, then disables API command source and obstacle avoidance. This is a good place to teach

defensive robotics programming: success and failure paths should converge on the same safe shutdown.

12. Velocity Commands Versus Increment Commands

Students often assume one motion API is simply “newer” than another, but the distinction is semantic. A velocity command is a stream of desired body-frame velocity. It must be repeated at a suitable rate because a single velocity command does not represent a complete path. An increment command asks for a bounded local displacement and yaw increment. It is closer to a leg in a patrol plan, but it is still not map navigation.

The repository helper `run_velocity_leg()` sends repeated `Move(vx, vy, vyaw)` calls until a duration expires. The helper `run_increment_leg()` sends `MoveToIncrementPosition(dx, dy, dyaw)` a small number of pulses, then waits for the motion to settle. The newer recommended pattern avoids flooding the increment command for the entire leg window while still making the command robust enough for the service to receive it.^{[10](#)}

Question	Velocity leg	Increment leg
What is specified?	Speed and duration.	Local displacement and yaw increment.
What API is used?	<code>Move(vx, vy, vyaw)</code> .	<code>MoveToIncrementPosition(dx, dy, dyaw)</code> .
How is it sent?	Repeated at a control rate for the duration.	Pulsed a few times, then allowed to settle.
Typical student mental model	“Walk forward slowly for 2 seconds.”	“Move about 0.3 m forward.”
Main risk	Forgetting to stop or sending too fast.	Treating local increments as global goals.

13. Sensor Capture and Inspection Evidence

The Day 2 sensor story is intentionally simple and useful. The primary evidence sensor is the front camera via `VideoClient.GetImageSample()` . State evidence comes from `rt/sportmodestate` . Optional platform probes can add additional context, but the core inspection deliverable is the camera image plus state log plus metadata.

The course materials emphasize that `VideoClient` is an RPC-style client in these examples, not a DDS video topic. The script pulls a JPEG/binary sample, decodes it using OpenCV, and writes it to disk. During a full patrol run, the integrated runner captures the starting

checkpoint and captures again after each leg at the leg's `to_checkpoint` . The camera does not steer the robot toward colored cones; it records what the robot saw when stopped.[11](#) [12](#)

Capture artifact	What it proves	What it does not prove
<code>frame.jpg</code>	A visual scene was recorded at a named checkpoint.	That the robot used vision to navigate.
<code>metadata.json</code> capture map	Which checkpoint IDs have captured frames.	That the physical cone was exactly reached.
<code>state_slice.jsonl</code>	Nearby state samples around the capture.	Full localization or mapped position.
Full <code>sportmodestate.jsonl</code>	Mode, gait, error code, velocity over time.	Semantic understanding of the environment.

Instructor prompt: “What can an inspector learn from the image that JSONL alone cannot provide?” Expected answers include obstacle presence, lighting, scene mismatch, wrong room, human safety issue, glare, or visible robot/camera alignment problems.

14. Integrated Patrol Runner: The Whole Pipeline in One Script

The integrated patrol runner is the best single-file narrative for the afternoon hardware workflow. It loads and optionally clamps the plan against scenario limits, creates a run folder, initializes sport, avoid, video, and state logging clients, prepares the robot posture, captures the starting checkpoint, enables obstacle avoidance, executes all plan legs, captures after each checkpoint, releases avoid mode, stops logging, writes metadata, and optionally runs the validator.[12](#) [10](#)

This structure shows why Day 2 is more than motion. If capture fails but motion succeeds, the run may be a partial inspection outcome. If validation fails, the motion demonstration is not yet a valid deliverable. If the plan violates scenario limits, clamping warnings explain what the script changed before motion.

Plain Text

```
Load scenario + plan
  → clamp motion limits
  → create run directory
  → initialize clients
  → stand + balance
  → start state logger
```

- capture cp_A
- enable avoid + API command source
- execute leg 1 → dwell → capture checkpoint
- execute leg 2 → dwell → capture checkpoint
- execute leg 3 → dwell → capture checkpoint
- release avoid + stop logger
- write metadata
- validate run folder

Runner phase	Failure mode	Instructor diagnosis
Plan load	Missing checkpoints or legs.	Validate JSON structure before hardware.
Scenario clamp	Leg value exceeds limits.	Teach why class caps override ambitious plans.
Client init	SDK import/network timeout.	Return to readiness and network checks.
Start capture	No camera frame.	Increase wait, check video service, use <code>--no-capture</code> only for motion debug.
Avoid enable	<code>SwitchSet</code> or <code>SwitchGet</code> fails.	Check mode, firmware, app state, and previous cleanup.
Leg execution	Stops short or overshoots.	Tune <code>dx</code> , <code>dyaw</code> , or <code>--leg-wait</code> .
Validation	Missing image, JSONL, or metadata field.	Treat validator output as a repair checklist.

15. Field Trial and Tuning

The field-trial lab teaches students to convert observations into controlled plan changes. The loop is observe → tune → trial → report. If the robot stops short, increase `dx` or increase leg wait. If it overshoots or approaches a wall, decrease `dx` . If the turn is too small, increase `dyaw` ; if too large, decrease it. These adjustments are still open-loop calibration, not camera-based steering.¹³

The repository provides `lab04_tune_plan.py` to copy a plan and apply leg overrides, and `lab04_field_trial.py` to run the tuned plan through the integrated runner and write `field_test.md` . This design reinforces a professional engineering habit: changes should be explicit, repeatable, and documented.

Observation	Likely tuning action	Report language
Robot stops short of cone B.	Increase <code>dx</code> slightly or increase <code>--leg-wait</code> .	“Baseline under-reached <code>cp_B</code> ; tuned leg 1 distance.”
Robot overshoots near wall.	Decrease final <code>dx</code> .	“Reduced final forward increment to preserve wall clearance.”
Turn at B is too small.	Increase turn-leg <code>dyaw</code> .	“Increased yaw increment to align with second corridor segment.”
Turn at B is too large.	Decrease <code>dyaw</code> .	“Reduced yaw increment after over-rotation.”
Capture is blurry.	Increase dwell or capture wait.	“Added settling time before checkpoint image.”

A strong field-trial report compares the baseline run folder with the tuned run folder, lists checkpoint outcomes as pass/partial/fail, references validator status, and states one next change. It should not claim success merely because the robot moved.

16. Optional Gazebo and ROS 2 Context

The optional Gazebo extension introduces simulation as a design and validation environment before field deployment. ROS describes ROS as a set of open-source software libraries and tools for building robot applications.¹⁴ ROS 2 Humble documentation demonstrates launching Gazebo simulations, bridging topics, publishing `geometry_msgs/Twist` velocity commands, visualizing sensor data, and moving a simulated robot.¹⁵ Gazebo’s own documentation explains that `ros_gz_bridge` exchanges messages between ROS 2 and Gazebo Transport, allowing commands and sensor data to move between the two ecosystems.¹⁶

In the Day 2 repository, the Gazebo extension uses ROS 2 and `/cmd_vel` , while the hardware patrol labs use `unitree_sdk2_python` over Ethernet. This distinction is pedagogically valuable. Simulation helps students understand timing, topic flow, and movement concepts, but it does not remove the need for hardware readiness, spotters, cones, speed limits, and cleanup.

Aspect	Gazebo extension	Physical Go2 patrol
--------	------------------	---------------------

Main interface	ROS 2 topic <code>/cmd_vel</code> with <code>geometry_msgs/Twist</code> .	Python SDK clients such as <code>ObstaclesAvoidClient</code> .
Robot required	No.	Yes.
Primary risk	Software setup, display, ROS environment.	Physical motion and field safety.
Evidence	Topic list, simulation screenshot, short motion result.	Valid <code>run_*</code> folder with images and state logs.
Lecture message	“Simulation helps reason before field deployment.”	“Hardware requires conservative execution and evidence.”

17. SLAM and Navigation: What Students Should and Should Not Claim

A central Day 2 teaching responsibility is vocabulary discipline. Unitree’ s SLAM/navigation documentation describes separate SLAM/navigation service interfaces and limits them to EDU robot dogs with expansion dock and official Unitree-purchased lidar versions, and it specifies constrained environments such as static indoor flat ground with rich features and area less than 25 m by 25 m.¹⁷ This supports the course distinction: the Python patrol labs do not automatically become SLAM simply because the robot has obstacle avoidance or lidar-related capabilities.

Students should therefore say: “Our Day 2 patrol used a local obstacle-avoidance service and scripted local increments from `patrol_plan.json` .” They should not say: “The robot mapped the room,” “The robot navigated to cones using vision,” or “The robot performed SLAM patrol,” unless the instructor has explicitly enabled and demonstrated the appropriate mapping/navigation stack.

Claim	Acceptable?	Correction
“The robot executed a local increment patrol under obstacle avoidance.”	Yes.	This accurately describes the Day 2 Python workflow.
“The camera captured evidence at checkpoints.”	Yes.	This is exactly what <code>VideoClient</code> contributes.
“The robot used the camera to steer to colored cones.”	No.	The camera records evidence; it does not steer in this lab.

“The patrol used GPS navigation.”	No.	Increments are local body-frame commands, not GPS goals.
“This is full SLAM.”	No.	SLAM requires mapping/localization services not used in the main Python patrol.
“Simulation used ROS 2 /cmd_vel ; hardware used SDK clients.”	Yes.	This is the correct sim/hardware contrast.

18. Capstone Presentation Expectations

The Day 2 capstone is short but important. Teams should present a five-minute demo that links scenario, architecture, evidence, and one failure/fix. The repository rubric focuses on safety, evidence, and technical use of avoid plus capture from the labs.[18](#)

A strong team presentation does not merely show a moving robot. It begins with the inspection scenario, shows the block diagram, identifies the planned checkpoints and motion limits, demonstrates or plays back the run, opens the run folder, shows at least one checkpoint frame, reports validator status, and explains one tuning or abort lesson.

Presentation section	What to show	What the instructor should listen for
Scenario	Arena, checkpoints, limits, abort rules.	The team understands bounded operation.
Architecture	PC, DDS/RPC, clients, robot, run folder.	The team can explain data and control flow.
Plan	patrol_plan.json legs and dwell times.	The team distinguishes increments from global goals.
Evidence	run_* folder, frame, JSONL sample, validator output.	The team values evidence over anecdote.
Failure and fix	Tuning note, abort condition, or capture issue.	The team can reason like field engineers.

19. Instructor Demonstration Script for a Three-Hour Lecture

The following sequence is suitable for an instructor-led demonstration without requiring every student machine to command hardware during the lecture. It emphasizes reading, prediction, and diagnosis before motion.

Demo	Command or artifact	Instructor narration
Readiness dry-run	<code>python course/day-02/lab-00/lab00_day2_readiness.py</code>	“Before motion, verify the machine and imports.”
Scenario creation	<code>--write-scenario my_team_scenario.json</code>	“This is the mission and safety contract.”
Scenario validation	<code>--validate-scenario my_team_scenario.json</code>	“Invalid mission definitions should fail early.”
Validate good fixture	<code>lab01_validate_run_folder.py ...sample_run_pass</code>	“This is what valid evidence looks like.”
Validate bad fixture	<code>lab01_validate_run_folder.py ...sample_run_incomplete</code>	“Use errors as a repair checklist.”
Avoid dry-run	<code>lab04_obstacle_avoid_intro.py en6 -dry-run</code>	“Dry-run before physical motion.”
Increment patrol dry-run	<code>lab02_increment_patrol.py en6 --dry-run</code>	“Predict each leg before running.”
Integrated runner dry-run	<code>lab03_patrol_runner.py en6 --dry-run</code>	“Now motion, logging, and capture are one pipeline.”
Field tuning	<code>lab04_tune_plan.py ... --set 1:dyaw:0.7</code>	“Tuning is documented plan editing.”
Final validation	<code>lab01_validate_run_folder.py run_*</code>	“A run is not complete until evidence validates.”

20. Common Misconceptions and Corrections

Students will often conflate autonomy layers. The lecture should repeatedly separate command execution, local avoidance, evidence capture, and navigation intelligence. These corrections should be made gently but firmly, because precise vocabulary prevents unsafe assumptions in the lab.

Misconception	Why it is wrong	Correct mental model
“Obstacle avoidance means the robot is autonomous.”	Avoidance is local and reactive; it does not define mission goals.	Autonomy requires goal definition, evidence, decision logic, and safety rules.
“A checkpoint image means vision guided the robot.”	The image is captured after stopping; it is not used for steering.	Vision is evidence capture in the main Day 2 labs.
“A larger <code>dx</code> finishes faster, so it is better.”	Larger increments reduce supervision margin and increase open-loop error.	Use small increments and tune from evidence.
“If the terminal says PASS, the inspection is complete.”	The run folder may still lack useful evidence or field interpretation.	PASS is necessary but not sufficient; inspect artifacts.
“Gazebo success means hardware is safe.”	Simulation omits many physical risks.	Sim reduces uncertainty; field rules still govern hardware.
“SLAM is any robot movement with sensors.”	SLAM specifically involves mapping and localization.	Day 2 mainly uses local avoidance and open-loop plans.

21. Knowledge Check

Use these questions in the final ten minutes or as a written handout. Students should answer in complete sentences, because the goal is conceptual precision rather than command memorization.

Question	Expected answer
What are the five stages of the Day 2 inspection architecture?	Sense, log, decide, act, report.
Why does <code>patrol_plan.json</code> exist separately from <code>metadata.json</code> ?	The plan defines intended checkpoints and legs; metadata records run identity, environment, mode, and artifacts.
What must happen before <code>ObstaclesAvoidClient.Move()</code> can control avoid-mode movement?	Avoidance must be enabled and <code>UseRemoteCommandFromApi(True)</code> must be set.
Why should <code>Move()</code> be sent repeatedly for velocity motion?	It represents a velocity stream over time, not a complete path by itself.

What is the difference between <code>leg-wait</code> and checkpoint dwell?	<code>leg-wait</code> allows the increment motion to settle; dwell is the stop time at a checkpoint before capture or next action.
Why is Day 2 patrol not SLAM patrol?	It does not build/use a global map and localize to global goals; it executes local scripted increments under avoidance.
What evidence should a team show in the capstone?	Scenario, architecture, plan, valid run folder or recording, checkpoint image, validator status, and one failure/fix.

22. Suggested Board Diagrams

The lecture benefits from three simple board diagrams. The first is the inspection pipeline. The second is the run folder tree. The third is the L-shaped cone course with `cp_A` , `cp_B` , `cp_C` , straight increment legs, and one yaw increment at the corner.

Diagram	What to draw	Teaching point
Pipeline	Sensors → logs → scenario → patrol → report.	Autonomy is a whole workflow.
Run folder	<code>metadata</code> , <code>plan</code> , <code>jsonl</code> , <code>checkpoints</code> .	Evidence must be structured and validated.
Cone course	<code>cp_A</code> forward to <code>cp_B</code> , turn, forward to <code>cp_C</code> .	Increments are local and tuned, not semantic navigation.
API lifecycle	Init → avoid on → API command → move → zero → release.	Safe cleanup is part of control.
Sim vs hardware	<code>/cmd_vel</code> vs SDK clients.	Simulation and hardware use different interfaces.

23. Closing Summary

Day 2 is the first point in the Vinci Unitree course where students can see a complete robotics workflow emerge from small primitives. The robot does not simply execute commands; it operates inside a scenario, with safety limits, a plan, state logs, camera evidence, validation, tuning, and reporting. That is the main lesson.

The instructor should close by reinforcing three principles. First, **bounded autonomy is still autonomy** when it is explicit, safe, and evidence-producing. Second, **local avoidance is not SLAM**, and students should use accurate language when describing their work. Third, **field robotics is an evidence discipline**: if a patrol cannot be reconstructed from its plan, logs, images, metadata, and report, then the team has not completed an inspection mission.

References

Footnotes

1. Vinci-Al-Analytics/vinci-unitree , course/day-02/README.md , “Day 2 — Go2 Autonomy & Sandbox Capstone.” ↵
2. Vinci-Al-Analytics/vinci-unitree , course/SCHEDULE-TTT.md , Day 2 schedule map. ↵
3. [Nav2 Documentation — Navigation Concepts](#). ↵
4. Vinci-Al-Analytics/vinci-unitree , course/day-02/lab-00/README.md , “Day 2 Readiness & Inspection Scenario.” ↵ ↵2 ↵3
5. Vinci-Al-Analytics/vinci-unitree , course/day-02/lab-01/README.md , “Run Folder Schema & Bundle Validation.” ↵
6. Vinci-Al-Analytics/vinci-unitree , course/day-02/lab-04/README.md , “Multi-Leg Patrol (Increment Goals).” ↵
7. [Unitree Go2 SDK Development Guide — Obtain SDK](#). ↵
8. [Unitree Go2 SDK Development Guide — Quick Start](#). ↵
9. [Unitree Go2 SDK Development Guide — Avoidance Services Interface](#). ↵
10. Vinci-Al-Analytics/vinci-unitree , course/day-02/go2_patrol_helpers.py , shared Day 2 patrol helper implementation. ↵ ↵2 ↵3
11. Vinci-Al-Analytics/vinci-unitree , course/day-02/lab-03/README.md , “Sensor Integration & Data Management.” ↵
12. Vinci-Al-Analytics/vinci-unitree , course/day-02/lab-05/README.md , “Integrated Patrol Runner & Checkpoint Capture.” ↵ ↵2
13. Vinci-Al-Analytics/vinci-unitree , course/day-02/lab-06/README.md , “Field Trial & Tuning.” ↵
14. [ROS — Robot Operating System](#), Open Robotics. ↵
15. [ROS 2 Humble Documentation — Setting up a robot simulation \(Gazebo\)](#). ↵
16. [Gazebo Documentation — Use ROS 2 to interact with Gazebo](#). ↵

17. [Unitree Go2 SDK Development Guide — SLAM and Navigation Services Interface.](#) ↩
18. `Vinci-AI-Analytics/vinci-unitree` , `course/day-02/lab-07/README.md` , “Team capstone & presentations.” ↩